

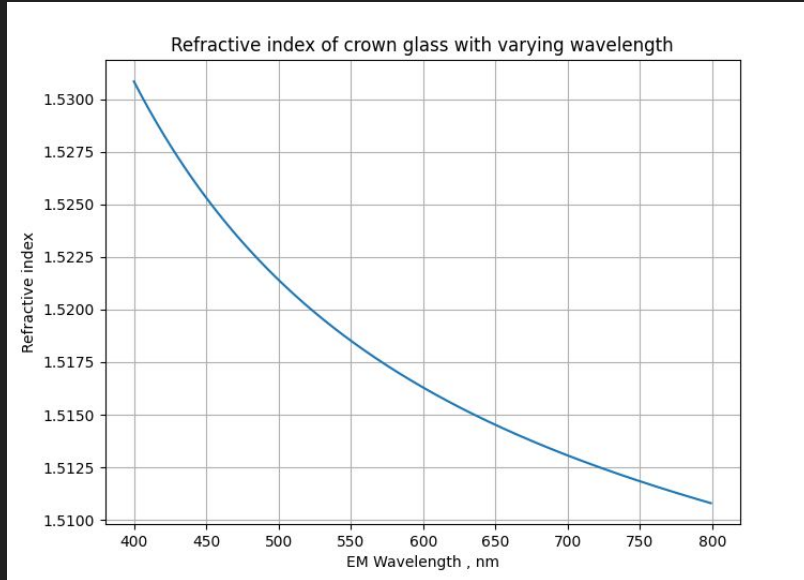
Bpho Computational Challenge 2025

By Ruben Fuguett-Gutierrez

Things to note

- All interactive plots will be shown at the end
- Due to time constraint and not being able to access the Challenge folder I did not finish all challenges
- All done in python
- I hope you enjoy!

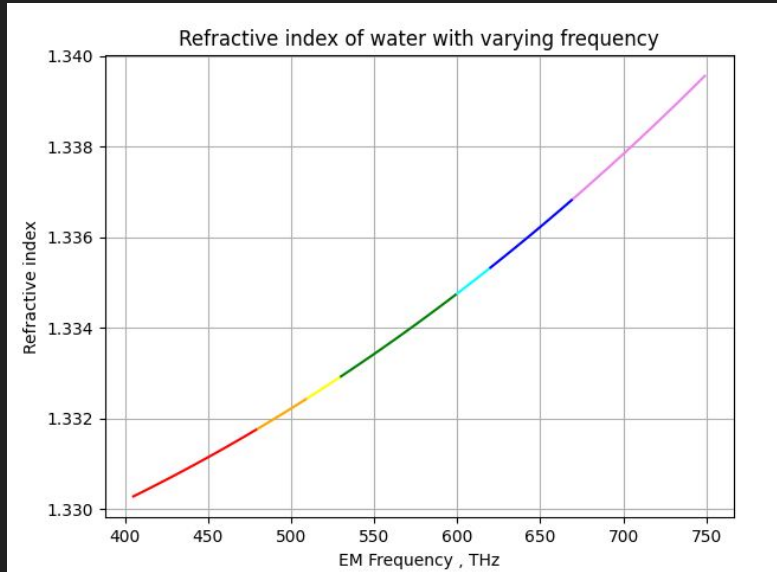
Task 1a - Plotting the refractive index of Crown glass depending on the wavelength of visible light



```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4
5 xpoints = np.arange(400, 800)
6 wl = xpoints/1000 #wl = wavelength in micrometers
7 ypoints = ((1+(1.03961212*wl**2)/(wl**2 - 0.00600069867)+
8 (0.231792344*wl**2)/(wl**2-0.0200179144)+(1.01046945*wl**2)/(wl**2 - 103.560653))**0.5)
9
10 plt.plot(xpoints, ypoints)
11 plt.xlabel("EM Wavelength , nm")
12 plt.ylabel("Refractive index")
13 plt.title("Refractive index of crown glass with varying wavelength")
14 plt.grid()
15 plt.show()
```

As Wavelength increases, the refractive index of the Crown glass decreases. This tells us that shorter wavelengths refract more than longer wavelengths (from 400-800nm)

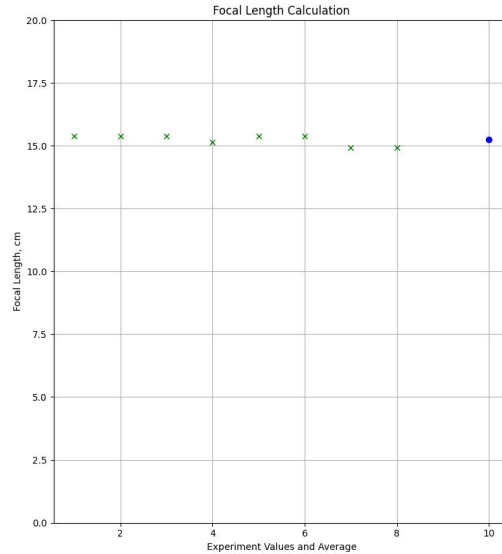
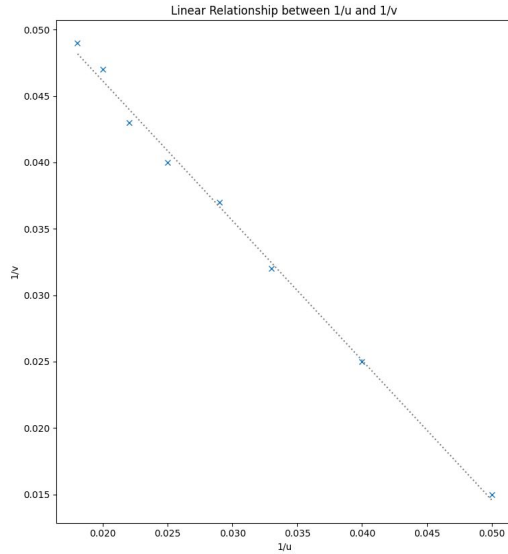
Task 1b - Plotting the refractive index of Water depending on the frequency of visible light



```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 freq_range = [
5     np.arange(405,480,2),
6     np.arange(480,510,2),
7     np.arange(510,530,2),
8     np.arange(530,600,2),
9     np.arange(600,620,2),
10    np.arange(620,670,2),
11    np.arange(670,750,2) ]
12
13 colors = np.array(["red","orange","yellow","green","cyan","blue","violet"])
14 for i in range(len(freq_range)):
15     index = ((1.731 - 0.261*(freq_range[i]/1000)**2)**(-0.5)+1)**0.5
16     plt.plot(freq_range[i],index,color=colors[i])
17
18
19 plt.xlabel("EM Frequency , THz")
20 plt.ylabel("Refractive index")
21 plt.title("Refractive index of water with varying frequency")
22 plt.grid()
23 plt.show()
```

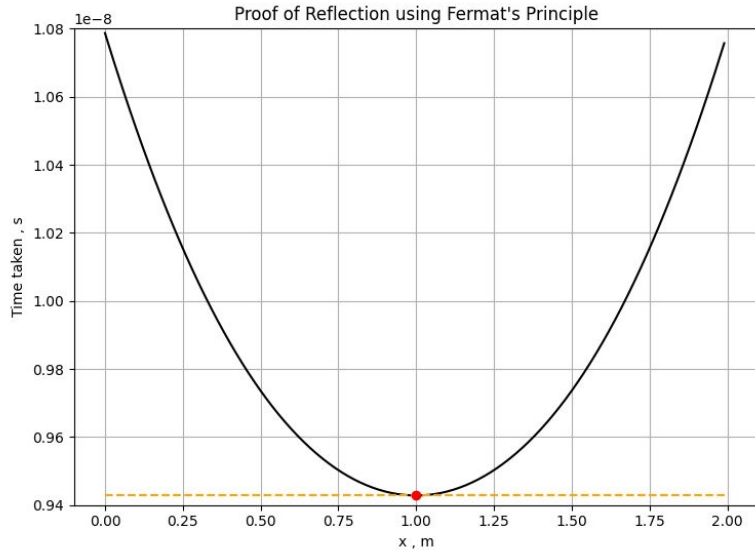
Similar to Task 1a, this shows that higher frequency waves refract more than low frequency waves. $F \propto 1/\lambda$

Task 2 - Calculating Focal Length of Lens



```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 plt.subplot(1,2,1) # graph 1. shows 1/u and 1/v
5 reciprocal_v = np.array([0.015, 0.025, 0.032, 0.037, 0.040, 0.043, 0.047, 0.049]) # y-axis
6 reciprocal_u = np.array([0.050, 0.040, 0.033, 0.029, 0.025, 0.022, 0.020, 0.018]) # x-axis
7
8 plt.plot(reciprocal_u, reciprocal_v, 'x') # points plotted
9
10
11 dy, intercept = np.polyfit(reciprocal_u, reciprocal_v,deg=1) # line of best fit calculation
12 line = dy*reciprocal_u + intercept
13
14 plt.plot(reciprocal_u, line, color="grey", linestyle="dotted") # line of best fit drawn
15
16 plt.xlabel("1/u")
17 plt.ylabel("1/v")
18 plt.title("Linear Relationship between 1/u and 1/v")
19
20 plt.subplot(1,2,2) # graph 2
21
22 reciprocal_f = reciprocal_u + reciprocal_v
23 f = 1/reciprocal_f
24 avg_f = sum(f)/8
25
26 x_axis = np.array([1,2,3,4,5,6,7,8])
27 plt.ylim(0,20) #range of y-axis
28 plt.plot(x_axis, f, "x", color="green")
29 plt.plot(10,avg_f,"o", color="blue")
30 plt.title("Focal Length Calculation")
31 plt.xlabel("Experiment Values and Average")
32 plt.ylabel("Focal length, cm")
33
34
35 plt.grid()
36 plt.show()
37 #standard deviation was 0.02
```

Task 3 - Using Fermat's last theorem to prove reflection

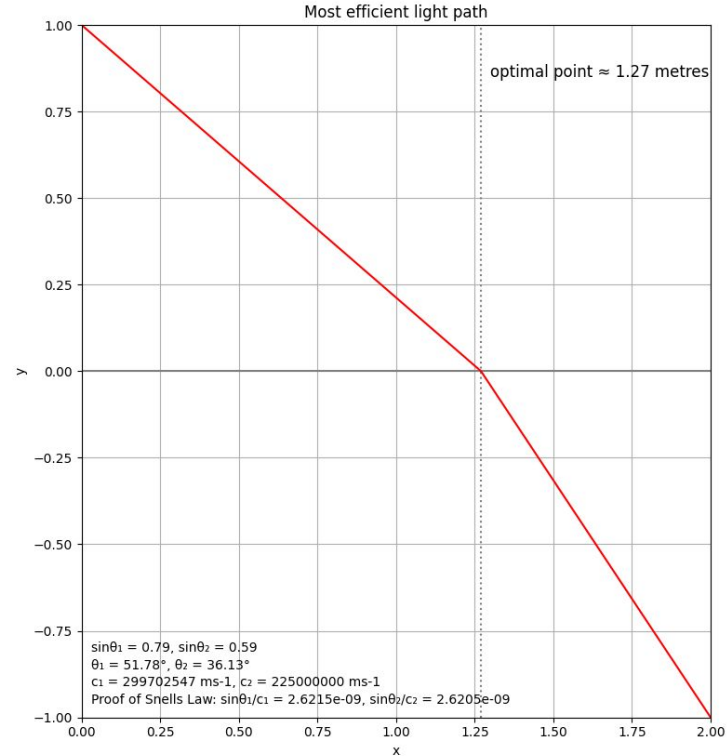
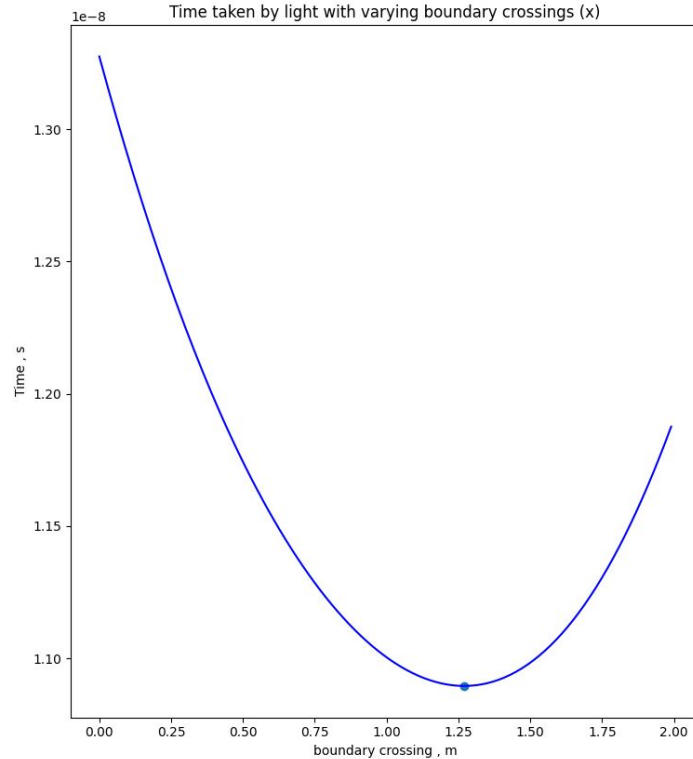


```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 #let A = (1,0) and L = (0,2)
5
6 c = 3*(10**8) #speed of light in ms^-1
7 n = 1 #refractive index
8 x = np.arange(0,2,0.01)
9 t = (x**2 + 1)**0.5/(c/n) + ((2-x)**2 + 1)**0.5/(c/n)
10 t_tangent = 2*((2)**0.5/(c/n)) #minimum value shown to be at x = 1
11
12 plt.xlabel("x , m")
13 plt.ylabel("Time taken , s")
14 plt.title("Proof of Reflection using Fermat's Principle")
15
16 plt.plot(x,t,"black")
17 plt.hlines(t_tangent,0,2, Linestyle="dashed",color="orange") # tangent line
18 plt.plot(1,t_tangent,"ro")
19 plt.ylim(0.94*10**(-8),1.08*10**(-8))
20 plt.grid()
21 plt.show()
22
23 #if time taken is always minimised by light, the minimum value of t must be taken by light
24 #since the minimum occurs at x=L/2 both paths are symmetrical and therefore light is reflected
```

Task 4 - Demonstrating a proof of Snell's Law

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 #let B = (2,-1)
5
6 c = 299792458 # in vacuum
7 c1 = 299702547 # in air
8 c2 = 225000000 # in water
9 x = np.arange(0,2,0.01)
10 t = (x**2 + 1)**0.5/c1 + ((2-x)**2 + 1)**0.5/c2
11 index_min = np.argmin(t) #locates minimum value of t
12 x_min = x[index_min] #calculates value of x needed for minimum value of t
13 t_min = t[index_min] #outputs minimum value of t
14
15 plt.subplot(1,2,1) #graph 1
16 plt.plot(x_min,t_min,"o") #plotting minimum point
17 plt.plot(x,t, "b") #plotting function
18
19 plt.xlabel("boundary crossing , m")
20 plt.ylabel("Time , s")
21 plt.title("Time taken by light with varying boundary crossings (x)")
22
23 plt.subplot(1,2,2) #graph 2
24
25 pathx = [0,x_min,2] #plotting path that light takes
26 pathy = [1,0,-1]
27 sin0 = x_min/(x_min**2+1)**0.5 #angle number 1
28 sin0 = (2-x_min)/((2-x_min)**2+1)**0.5 #angle number 2
29 0 = np.rad2deg(np.arcsin(sin0))
30 0 = np.rad2deg(np.arcsin(sin0))
31
32 plt.xlabel("x")
33 plt.ylabel("y")
34 plt.title("Most efficient light path")
35 plt.text(x_min + 0.03,0.85,"optimal point ≈ " + str(round(x_min, 2)) + " metres", size=12) #top left text
36 plt.text(0.03, -0.81,"sin01 = " + str(round(sin0,2)) + ", sin02 = " + str(round(sin0,2)), size=10) #sintheta
37 plt.text(0.03, -0.86,"θ1 = " + str(round(0,2)) + "°, θ2 = " + str(round(0,2)) + "°", size=10) #angles
38 plt.text(0.03, -0.91,"c1 = " + str(c1) + " ms-1, c2 = " + str(c2) + " ms-1", size=10) #speeds of light
39 plt.text(0.03, -0.96,"Proof of Snells Law: sin01/c1 = " + str(round((sin0/c1), 13)) + ", sin02/c2 = " + str(round((sin0/c2),13)))
40
41 plt.vlines(x_min,-1,1, linestyle="dotted", color="grey")
42 plt.ylim(-1,1)
43 plt.hlines(0,0,2, color="grey")
44 plt.xlim(0,2)
45 plt.plot(pathx, pathy, "r")
46
47 plt.grid()
48 plt.show()
```

Graphs plotted for task 4



Task 5 - reflection of an image (interactive)

```
1 from matplotlib import pyplot as plt
2 import numpy as np
3
4 x, y = 0, 0
5
6 image = plt.imread(r'C:\Users\igula\OneDrive\Desktop\Python projects\Bpho Computing Challenge 2025\ingot2.png')
7 mirror_image = np.fliplr(image) #flipped image
8
9 fig, ax = plt.subplots()
10 img1 = ax.imshow(image, extent=[x, x + 225, y, y + 225])
11 img2 = ax.imshow(mirror_image, extent=[-x - 225, -x, y, y + 225])
12
13 ax.set_title("Reflected Image")
14 ax.set_xlim(-450,450) #boundaries
15 ax.set_ylim(0,450)
16 ax.vlines(0,0,450, linestyle="dotted", color="green") #mirror line
17
18 def on_key(event):
19     global x, y
20     if event.key == 'w':
21         y += 3
22     elif event.key == 'x':
23         y -= 3
24     elif event.key == 'a':
25         if x >= 3:
26             x -= 3
27     elif event.key == 'd':
28         x += 3
29     else:
30         return # event for when keys are pressed
31     img1.set_extent([x, x + 225, y, y + 225]) #changes values of image
32     img2.set_extent([-x - 225, -x, y, y + 225])
33     fig.canvas.draw_idle() # draws to the screen
34
35 fig.canvas.mpl_connect('key_press_event', on_key) #if any key is pressed, the on_key function is called and executed WAXD give movements
36 plt.axis('off')
37 plt.show()
```

Task 6/7 - Projection through converging lens (interactive)

```
1 from matplotlib import pyplot as plt
2 import numpy as np
3
4 F, x, y = 60, 100, 100
5 X = -(F/(x-F))*x #projected image
6 Y = X*(y/x)
7
8 image = plt.imread(r'C:\Users\lgula\OneDrive\Desktop\Python projects\Bpho Computing Challenge 2025\ingot.png')
9 projection = np.fliplr(np.flipud(image))
10
11 fig, ax = plt.subplots()
12 img1 = ax.imshow(image, extent=[x, x + 30, y, y + 30])
13 img2 = ax.imshow(projection, extent=[X, X + 30, Y, Y + 30])
14
15 ax.plot(F,0,"-",color="red")
16 ax.plot(F,0,"-",color="red")
17 ax.set_xlim(-200,200)
18 ax.set_ylim(-200,200)
19 ax.set_title("Projection of an image through a converging lens, F=60cm")
20
21 def on_key(event):
22     global x, y, X, Y
23     if event.key == 'w':
24         y += 2
25     elif event.key == 'x':
26         y -= 2
27     elif event.key == 'a':
28         if x > 0:
29             x -= 2
30     elif event.key == 'd':
31         x += 2
32     else:
33         return
34     if F < x:
35         X = -(F/(x-F))*x
36         Y = X*(y/x)
37         img2.set_data(np.fliplr(np.flipud(image)))
38     else:
39         X = (F/(x-F))*x
40         Y = (y/x)*X
41         img2.set_data(image)
42     img1.set_extent([x, x + 30, y, y + 30])
43     img2.set_extent([X, X + 30, Y, Y + 30])
44
45     fig.canvas.draw_idle()
46
47 fig.canvas.mpl_connect('key_press_event', on_key)
48
49 plt.xlabel("Distance, cm (x)")
50 plt.ylabel("Distance, cm (y)")
51 plt.vlines(0,-200,200, linestyle="dotted",color="blue")
52 plt.show()
```

Task 8-10

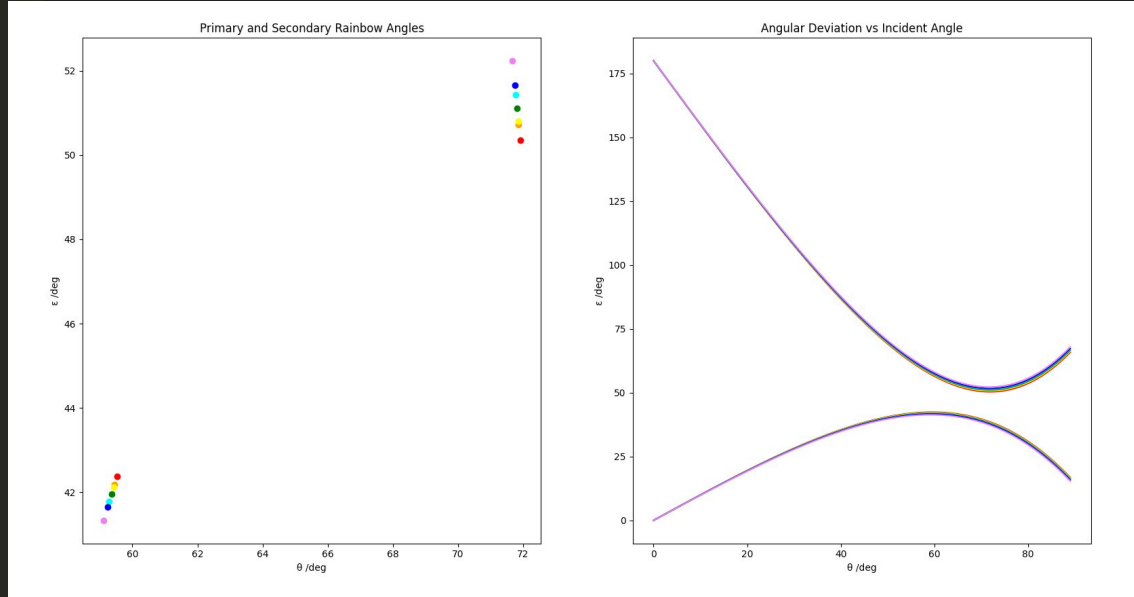
Unfortunately, I didn't get a chance to complete challenge 8 and i couldn't start challenges 9 and 10.

Initial Task 8 Code

```
1 from matplotlib import pyplot as plt
2 import numpy as np
3 from PIL import Image as img
4 from scipy.ndimage import map_coordinates
5
6 fig, ax = plt.subplots()
7
8 radius = 100
9
10 image = np.array(img.open(r'C:\Users\igula\OneDrive\Desktop\Python projects\Bpho Computing Challenge 2025\ingot.png').convert('RGB'))
11 h, w, _ = image.shape
12
13 x_location, y_location = np.meshgrid(np.arange(w), np.arange(h)) #makes pixel array
14
15 theta = np.arctan(y_location/np.sqrt(radius**2-x_location**2))
16 m = np.tan(2*theta)
17 x_projection = -(m*np.sqrt(radius**2-y_location**2)-y_location)/((y_location/x_location)+m)
18 y_projection = x_projection*(y_location/x_location)
19
20 coords = np.stack([y_projection,x_projection])
21
22
23
24 def on_key(event):
25     global x_location,y_location,x_projection,y_projection
26     if event.key == 'w':
27         y_location += 2
28     elif event.key == 'x':
29         y_location -= 2
30     elif event.key == 'a':
31         if x_location > 0:
32             x_location -= 2
33     elif event.key == 'd':
34         x_location += 2
35     else:
36         return
37     #redo calculation for each pixel
38     fig.canvas.draw_idle()
39
40 fig.canvas.mpl_connect('key_press_event', on_key)
```

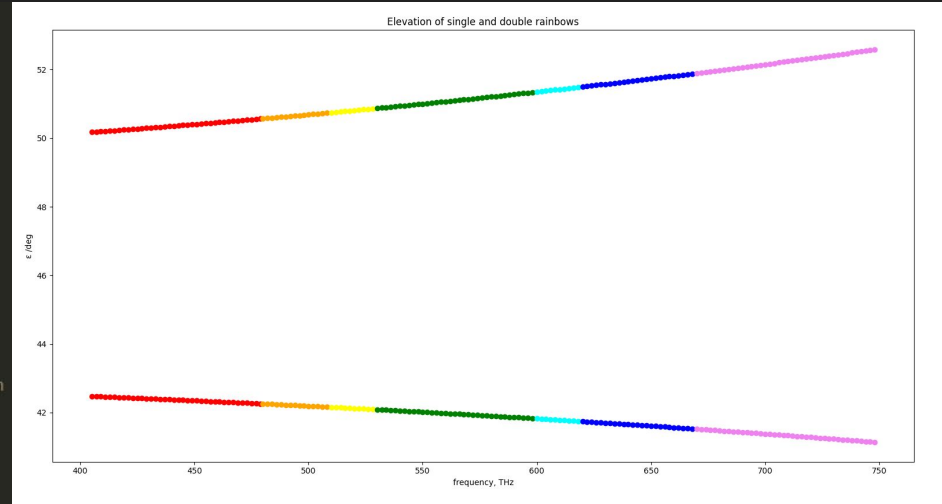
Task 11a - Plots showing elevation against incident angle

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 freq = np.array([440,505,520,565,610,640,710]) #average frequencies for colors. from wikipedia
5 colors = np.array(["red","orange","yellow","green","cyan","blue","violet"])
6
7 index = (((1.731 - 0.261*(freq/1000)**2)**(-0.5)+1)**0.5)
8
9 deg = 180/np.pi # conversion factor for radians into degrees
10 safe_arcsin = lambda x: np.arcsin(np.clip(x, -1, 1)) #arcsin turned into inverse sin
11
12 θ1 = safe_arcsin(np.sqrt((9 - index**2)/8))*deg #visible rainbow values to plot on graph 1
13 θ2 = safe_arcsin(np.sqrt((4 - index**2)/3))*deg
14
15 ε1 = 180 - 6*safe_arcsin(np.sin(np.radians(θ1))/index)*deg + 2*θ1
16 ε2 = 4*safe_arcsin(np.sin(np.radians(θ2))/index)*deg - 2*θ2
17
18 θ = np.arange(0,90,1) # all values of θ used for graph 2
19
20 ε3 = []
21 ε4 = []
22
23 for n in index:
24     ε3.append(180 - 6*safe_arcsin(np.sin(np.radians(θ))/n)*deg + 2*θ)
25     ε4.append(4*safe_arcsin(np.sin(np.radians(θ))/n)*deg - 2*θ)
26
27 plt.subplot(1,2,1) #graph 1
28 plt.xlabel("θ /deg")
29 plt.ylabel("ε /deg")
30 plt.title("Primary and Secondary Rainbow Angles")
31
32 for i in range(len(colors)):
33     plt.scatter(θ1[i], ε1[i],color=colors[i]) #plots points for rainbow angles
34     plt.scatter(θ2[i], ε2[i],color=colors[i])
35
36 plt.subplot(1,2,2) #graph 2
37 plt.xlabel("θ /deg")
38 plt.ylabel("ε /deg")
39 plt.title("Angular Deviation vs Incident Angle")
40
41 for i in range(len(colors)):
42     plt.plot(θ, ε3[i],color=colors[i]) #plots theta and epsilon
43     plt.plot(θ, ε4[i],color=colors[i])
44
45 plt.show()
```



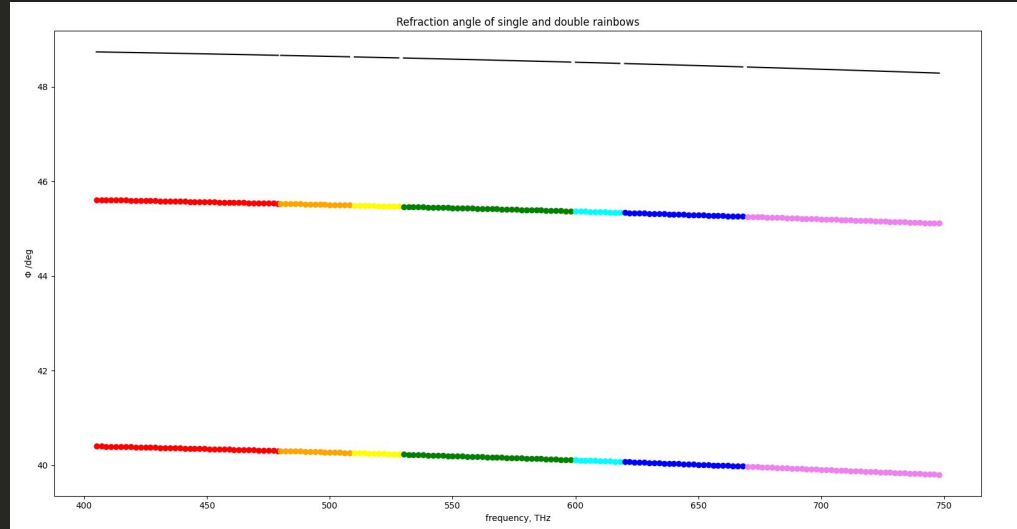
Task 11b - Elevation of rainbow depending on frequency

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 freq_range = [
5     np.arange(405,480,2),
6     np.arange(480,510,2),
7     np.arange(510,530,2),
8     np.arange(530,600,2),
9     np.arange(600,620,2),
10    np.arange(620,670,2),
11    np.arange(670,750,2) ] #average frequencies for colors. from wikipedia
12
13 colors = np.array(["red","orange","yellow","green","cyan","blue","violet"])
14
15 deg = 180/np.pi # conversion factor for radians into degrees
16 safe_arcsin = lambda x: np.arcsin(np.clip(x, -1, 1)) #arcsin turned into inverse sin
17
18 for i in range(len(colors)):
19     freq = freq_range[i]
20     index = (((1.731 - 0.261*(freq/1000)**2)**(-0.5)+1)**0.5)
21
22     θ1 = safe_arcsin(np.sqrt((9-index**2)/8))*deg #visible rainbow values to plot on graph 1
23     θ2 = safe_arcsin(np.sqrt((4-index**2)/3))*deg
24
25     ε1 = 180 - 6*safe_arcsin(np.sin(np.radians(θ1))/index)*deg + 2*θ1
26     ε2 = 4*safe_arcsin(np.sin(np.radians(θ2))/index)*deg - 2*θ2
27     plt.plot(freq,ε1,"o",color=colors[i],)
28     plt.plot(freq,ε2,"o",color=colors[i])
29
30 plt.xlabel("frequency, THz")
31 plt.ylabel("ε /deg")
32 plt.title("Elevation of single and double rainbows")
33 plt.show()
```



Task 11c - Angle of refraction and critical angle

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 freq_range = [
5     np.arange(405,480,2),
6     np.arange(480,510,2),
7     np.arange(510,530,2),
8     np.arange(530,600,2),
9     np.arange(600,620,2),
10    np.arange(620,670,2),
11    np.arange(670,750,2) ]
12
13 colors = np.array(["red","orange","yellow","green","cyan","blue","violet"])
14
15 deg = 180/np.pi # conversion factor for radians into degrees
16 safe_arcsin = lambda x: np.arcsin(np.clip(x, -1, 1)) #arcsin turned into inverse sin
17
18 for i in range(len(colors)):
19     freq = freq_range[i]
20     index = (((1.731 - 0.261*(freq/1000)**2)*(-0.5)+1)**0.5)
21
22     θ1 = safe_arcsin(np.sqrt((9 - index**2)/8))*deg
23     θ2 = safe_arcsin(np.sqrt((4 - index**2)/3))*deg
24     θ1 = safe_arcsin(np.sin(np.radians(θ1))/index)*deg
25     θ2 = safe_arcsin(np.sin(np.radians(θ2))/index)*deg
26     θcrit = safe_arcsin(1/index)*deg
27
28     plt.plot(freq,θ1,"o",color=colors[i])
29     plt.plot(freq,θ2,"o",color=colors[i])
30     plt.plot(freq,θcrit,color="black")
31
32 plt.xlabel("frequency, THz")
33 plt.ylabel("θ /deg")
34 plt.title("Refraction angle of single and double rainbows")
35 plt.show()
```



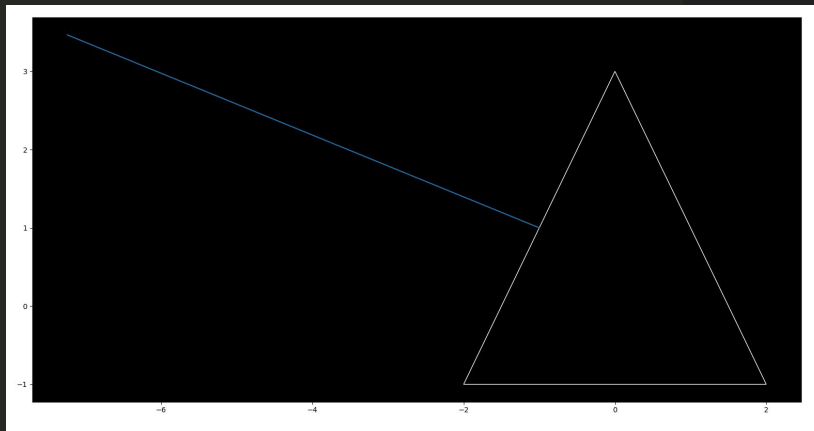
Task 11d - Primary and Secondary rainbows (interactive)

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 freq_range = [
5     np.arange(405,480,2),
6     np.arange(480,510,2),
7     np.arange(510,530,2),
8     np.arange(530,600,2),
9     np.arange(600,620,2),
10    np.arange(620,670,2),
11    np.arange(670,750,2) ]
12
13 colors = np.array(["red","orange","yellow","green","cyan","blue","violet"])
14
15 deg = 180/np.pi # conversion factor for radians into degrees
16 safe_arcsin = lambda x: np.arcsin(np.clip(x, -1, 1)) #arcsin turned into inverse sin
17 alpha = 40
18 distance = 10
19 h = 5
20
21 fig, ax = plt.subplots()
22
23 def on_key(event): #control radius and alpha
24     global alpha, distance
25     if event.key == 'a':
26         alpha -= 1
27     elif event.key == 'd':
28         alpha += 1
29     elif event.key == 'x':
30         distance -= 1
31     elif event.key == 'w':
32         distance += 1
33     else:
34         return
35     draw_rainbow()
36
37 def draw_rainbow():
38     ax.clear()
39     for i in range(len(colors)):
40         freq = freq_range[i]
41         index = (((1.731 - 0.261*(freq/1000)**2)**(-0.5)+1)**0.5)
42
43         theta1 = safe_arcsin(np.sqrt((9-index**2)/8))*deg
44         theta2 = safe_arcsin(np.sqrt((4-index**2)/3))*deg
45         e1 = 180 - 6*safe_arcsin(np.sin(np.radians(theta1))/index)*deg + 2*theta1
46         e2 = 4*safe_arcsin(np.sin(np.radians(theta2))/index)*deg - 2*theta2
47         r_radius1 = distance*np.sin(np.radians(e1))*np.cos(np.radians(alpha)) #radius of rainbow 1
48         r_radius2 = distance*np.sin(np.radians(e2))*np.cos(np.radians(alpha)) #radius of rainbow 2
49         r_limit1 = r_radius1 - distance*np.sin(np.radians(e1-alpha)) - h
50         r_limit2 = r_radius2 - distance*np.sin(np.radians(e2-alpha)) - h
51
52         theta = np.linspace(0, np.pi, 500) # angles for upper semicircle (0 to 180 degrees)
53
54         for j in range(len(r_radius1)):
55             r1 = r_radius1[j]
56             r2 = r_radius2[j]
57             limit1 = r_limit1[j]
58             limit2 = r_limit2[j]
59             x1_uncut = r1 * np.cos(theta)
60             y1_uncut = r1 * np.sin(theta)
61             x2_uncut = r2 * np.cos(theta)
62             y2_uncut = r2 * np.sin(theta)
63
64             mask1 = y1_uncut >= limit1
65             mask2 = y2_uncut >= limit2
66
67             ax.plot(x1_uncut[mask1], y1_uncut[mask1], color=colors[i])
68             ax.plot(x2_uncut[mask2], y2_uncut[mask2], color=colors[i])
69         ax.set_title(f"α = {alpha}°, distance = {distance}")
70         plt.draw()
71 fig.canvas.mpl_connect('key_press_event', on_key)
72 draw_rainbow()
73 plt.title("Rainbow")
74 plt.show()
```

Task 12a

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 from matplotlib.patches import Polygon
4
5 def rotation(vector,angle):
6     R = np.array([
7         [np.cos(angle), -np.sin(angle)],
8         [np.sin(angle),  np.cos(angle)] #rotation transformation
9     ])
10    return R @ vector
11
12 safe_arcsin = lambda x: np.arcsin(np.clip(x, -1, 1)) #sin-1
13
14 x = -7 #arbitrary value needed for vector for  $\theta_i$  calculation
15  $\theta_i$  = 5 #variable that will be changed
16 theta_i = np.radians( $\theta_i$ )
17
18 x2 = 7 #arbitrary value for refracted ray1
19 v2 = np.array([x2+1,-0.5-0.5*x2])
20
21
22 wavelength_range = [
23     np.arange(380,450,5),
24     np.arange(450,485,5),
25     np.arange(485,500,5),
26     np.arange(500,565,5),
27     np.arange(565,590,5),
28     np.arange(590,625,5),
29     np.arange(625,750,5) ]
30
31 colors = np.array(["violet","blue","cyan","green","yellow","orange","red"])
32
33 for i in range(len(wavelength_range)):
34     w1 = wavelength_range[i]/1000
35     index = ((1+(1.03961212*w1**2)/(w1**2 - 0.00600069867)+
36         (0.231792344*w1**2)/(w1**2-0.0200179144)+(1.01046945*w1**2)/(w1**2 - 103.560653))**0.5) #calculations taken from challenge 1a
```

```
38      $\theta_r$  = safe_arcsin(np.sin(theta_i/index))
39      $\theta_r\_rad$  = np.radians( $\theta_r$ )
40     u2 = rotation(v2, $\theta_r\_rad$ )
41     u2 = (u2[0+2*i]-1,u2[1+2*i]+1)
42     boundary2_x = np.arange
43
44
45 fig, ax = plt.subplots()
46
47 triangle_points = [(-2,-1),(2,-1),(0,3)]
48 triangle = Polygon(triangle_points, edgecolor="white", facecolor="none")
49 ax.add_patch(triangle)
50
51 v = np.array([x+1,-0.5-0.5*x]) #displace vector to origin to allow for rotation , equation of normal:  $y = -1/2x + 1/2$ 
52 u = rotation(v,theta_i)
53 u = ([u[0]-1,u[1]+1])
54
55 P1 = np.array([-1,1])
56
57 ray_x = np.array([u[0],-1])
58 ray_y = np.array([u[1], 1])
59
60 ax.plot(ray_x,ray_y)
61 ax.set_facecolor("black")
62 plt.show()
```



I hope you enjoyed my take on the challenges presented!

Any suggestions on what to do better would be greatly appreciated, this is my first time doing a long-form project like this and it was really fun to do. Although it was not fully complete, I learnt a lot of different tools used in programming and a lot of effort went into doing research for each task.